Chapter 9

Building a Micnet Network

9.1 Introduction

A Micnet network allows communication between two or more independent XENIX systems. The network consists of computers connected by serial communication lines (RS-232 ports connected by cable). Each computer in the network runs as an independent system, but allows users to communicate with the other computers in the network through the **mail**, **rcp**, and **remote** commands. These commands pass information such as mail, files, and even other commands from one computer to another.

It is the system manager's task to build and maintain a Micnet network. The system manager decides how to connect the computers, makes the actual physical connections, then uses the **netutil** program to define and start the network.

This chapter explains how to plan a network and then build it with the **netutil** program. In particular, it describes:

- How to choose machine names and aliases
- How to draw the network topology map
- How to assign serial lines
- How to create the Micnet files
- How to distribute the Micnet files
- How to test the Micnet network

9.2 Planning a Network

To build a Micnet network, you must give the **netutil** the names of the computers that will be in the network, a description of how you will connect the computers, a list of the serial lines you will use, the names of the users who will use the network, and what aliases (if any) they will be known by.

To keep the task as simple as possible, you should take some time to plan the network and make lists of the information you will be required to supply. To help you make these lists, the following sections suggest ways to plan a network.

9.2.1 Choosing Machine Names

A Micnet network requires that each computer in the network have a unique *machine name*. A machine name helps distinguish each computer from other computers in the network. It is best to choose machine names as the first step in planning the network. This prevents confusion later on when you build the network with the **netutil** program.

A machine name should suggest the location of the computer or the people who are users on the computer; however, you may use any name you wish. The name must be unique and consist of letters and digits. The Micnet programs use only the first eight characters of each name, so be sure those characters are unique.

The **netutil** program saves the machine name of a computer in a */etc/systemid* file. One file is created for each computer. After you have built and installed the network, you can find out the machine name of the computer you are using by displaying the contents of this file.

9.2.2 Choosing a Network Topology

The network topology is a description of how the computers in the network are connected. In any Micnet network, there are two general topologies from which all topologies can be constructed. These are *star* and *serial*.

In a star topology, all computers are directly connected to a central computer. All communication passes through the central computer to the desired destination.

In a serial topology, the computers form a chain, with each computer directly connected to no more than two others. All communication passes down the chain to the desired destination.

A network may be strictly star, strictly serial, or a combination of star and serial topologies. The only restriction is that no network may form a ring. For example, you cannot close up a serial network by connecting the two computers at each end.

The kind of topology you choose depends on the number of computers you have to connect, how quickly you want communications to proceed, and how you want to distribute the task of passing along communications. A star topology provides fast communication between computers, but requires both a large portion of the central computer's total operation time and a large number of serial lines on the central computer. A serial topology distributes the communication burden evenly, requiring only two serial lines per computer, but it is slow if the chain is very long (communication between computers can take several minutes). Often a combination of star and serial topologies makes the best network. In any case, make the choice you think best. If you discover you have made a wrong choice, you may change the network at any time.

9.2.3 Drawing a Network Topology Map

A network topology map is a sketch of the connections between computers in the network. You use the map to plan the number and location of the serial lines you use to make the network.

You can make the map while you work out the topology. Simply arrange the machine names of each computer in the network on paper, then mark each pair of computers you wish to connect with serial lines. For example, the topology map for three computers might look like this:

```
a ---------- b ---------- c
```

As you draw, make sure that there is no more than one connection between any two computers in the network. Furthermore, make sure that no rings are formed (a ring is a series of connections that form a closed circle). Multiple connections and rings are not permitted.

9.2.4 Assigning Lines and Speeds

Once you have made the topology map, you can decide which serial lines to use. Since every connection between computers in the network requires exactly two serial lines (one on each computer), you need to be very careful about assigning the lines. Follow these steps:

1. Make a list of the serial lines (TTY lines) available for use on each computer in the network. You can see a list of the serial lines on a computer by displaying the file */etc/ttys*. A line is available if it is not connected to any device such as a terminal or modem.

2. Using the topology map, pick a computer, then assign one, and only one, serial line to each connection shown for that computer. The serial lines must be from the list of available lines for that computer. No line may be assigned more than once. For example, if computer *a* has only one available serial line (tty00), then the topology map should look like this:

```
  a ---------- b ---------- c
tty00
```

3. Repeat step 2 for all computers in the topology map. Make sure that each connection is assigned a line and that no two connections on any given computer have the same line. When finished, the map should look like this:

```
    a ---------- b ---------- c
tty00  tty02  tty03   tty04
```

   If a computer does not have enough available serial lines to meet its needs, you can make the lines available by removing the devices already connected to them. If you cannot remove devices, you must redraw your topology map.

4. Using the topology map, assign a serial line transmission speed for each computer pair. The speed may be any in the normal range for XENIX serial lines (typically 110 to 9600). Transmission speeds are a matter of preference. In general, a higher speed means a smaller amount of time to complete a transmission, but a greater demand on system's input and output capabilities. In some cases, transmission speeds are a matter of hardware capabilities. Some hardware is not capable of transmission speeds greater than 1200 baud. For this reason, 1200 is the recommended speed when first installing Micnet. You may then increase the speed if you find the hardware can support it.

5. After the topology map is completely filled in, make a list of all computer pairs, showing their machine names, serial lines, and transmission speeds. You will use this list when installing the network.

9.2.5 Choosing Aliases

Once you have decided how to connect the computers in the network, you can choose *aliases* for users in the network. An alias is a simple name that represents both a location (computer) and a user. Aliases are used by the **mail** command to allow you to refer to specific computers and users in a network without giving the explicit machine and user names. Although not a required part of the network, aliases can make the network easier to use and maintain.

There are three kinds of aliases: standard, machine, and forward. A standard alias is a name for a single user or a group of users. A machine alias is a name for a computer or an entire network (called a site). A forward alias is a temporary alias for a single user or group of users. A forward alias allows users who normally receive network communications at one computer to receive them at another.

When you build a network with the **netutil** program, you will be asked to provide standard aliases only. (You can incorporate machine and forward aliases into the network at your leisure.) Each standard alias must have a unique name and a list of the login names of the users it represents. You may choose any name you wish as long as it consists of letters and numbers, begins with a letter, and does not have the same spelling as the login names. The name should suggest the user or group of users it represents. The login names must be the valid login names of users in the network.

To help you prepare the aliases for entry during the **netutil** program, follow these steps:

1. Make a list of the user aliases (that is, the aliases that refer to just one user) and the corresponding login names of each user.

2. Make a separate list of the group aliases (that is, the aliases that refer to two or more users) and the login names or user aliases (from the first list) of the corresponding users. A group alias may have any number of corresponding users.

Note that there are a number of predefined group aliases. The name **all** is the predefined alias for all users in the network. The machine names of the computers in the network are predefined aliases for the users on each computer. Do not use these names when defining your own aliases.

9.3 Building a Network

You build a network with the **netutil** program. The program allows you to define the machines, users, and serial lines that make up the network.

To build a network, you must first create the Micnet files that define the network and then transfer these files to each computer in the network. After each computer receives the files, you may start the network and use it to communicate between computers.

The following sections describe how to build the network.

9.3.1 Creating the Micnet Files

The Micnet files are created with the **install** option of the **netutil** program. The **install** option asks for the names, aliases, and serial lines of each computer in the network. As you supply the information, it automatically creates the files needed for each computer. These files can then be transferred to the other computers in the network with the **save** and **restore** options of **netutil**. This means you can build the entire network from just one computer.

To use the **install** option, follow these steps:

1. Log in as the super-user.

2. Type:

   netutil

   and press the RETURN key. The program displays the network utility menu. The **install** option is the first item in the menu.

3. Type the number 1 and press the RETURN key. The program displays the following message:

```
Compiling new network topology
Overwrite existing network files? (yes/no)?
```

   Type *y* (for "yes") and press the RETURN key to overwrite the files. You must overwrite the existing network files to create the new network. The first time you install the network, these files contain default information that need not be saved. If you install the system a second time or expand the system, it may be wise to save a copy of these files before starting the **install** option. The files can be saved on a floppy disk with the **save** option, described later in Section 9.3.2 of this chapter.

   Once you have typed *y*, the program displays the following message:

```
Enter the name of each machine
       (or press RETURN to continue installation)
  Machine name:
```

4. Enter a machine name by typing the name and press the RETURN key. You may enter more than one name on a line by separating each with a comma (,) or a space. After you have entered all the names, press the RETURN key to continue to the next step. The program displays the names you have entered and asks if you wish to make changes:

```
Machine names are:  name1 name2 ... namen
Do you wish to make any changes?  (yes/no)?
```

5. Type *y* (for "yes") if you wish to change the machine names. Otherwise, type *n* (for "no") or just press the RETURN key to move on to the next step. If you type *n*, the program displays the message:

```
For each machine, enter the names of the machines
       to be connected with it
Machine name:
  Connect to:
```

6. Using the list of machine pairs you created when planning the network, enter the machine names of the computers that connect to the computer specified by *name*. You may enter more than one name on a line by separating each name with a comma (,) or a space. When you have entered the machine names of all computers that connect to the specified computer, press the RETURN key.

7. The program prompts you to supply the machine names of the computers that connect to the next computer, and continues with this process until you have given the connections for all the machine names you entered in step 3. Follow the procedure described in step 6 for each machine name. As the program asks for each new set of connections, it will show a list of the machine names it already knows are connected with the current computer. You need not enter these names. The program automatically checks for loops. If it finds one, it ignores the machine name that creates the loop and asks for another.

   Finally, when you have given the connections for all computers in the network, the program displays a list of the connections and asks if you wish to make corrections:

   ```
   network connections completed.

   Machine pairs are:
          name1  name2
          name1  name3
          name3  name4

   Do you wish to make any changes (yes/no)?
   ```

8. Type *y* if you wish to enter the connections again. Otherwise, type *n* to move to the next step. If you type *y*, the program displays the message:

   ```
   For each machine pair, enter the tty name and tty speeds
   For the name1 <==> name2 machine pair.
     Tty on name1:
   ```

9. Using the list of serial line assignments you created when planning the network, type the serial line name or number (for example, tty03 or 3) for the first computer in the pair and press the RETURN key. The program displays the message:

   ```
   Tty on name2:
   ```

10. Type the serial line name for the second computer in the pair and press the RETURN key. The program displays the message:

```
Speed:
```

11. Type the speed (for example, 1200) and press the RETURN key.

12. The program prompts you to supply the serial lines and transmission speeds of the next pair, and continues with this process until you have given this information for all the remaining machine pairs. Repeat steps 8 through 11 for each machine pair. When you have given serial lines and speeds for all pairs, the program displays this information and asks if you wish to make corrections:

```
Machine pairs are:

        name1  ttynn  name2  ttynn  speed
        name1  ttynn  name3  ttynn  speed
        name3  ttynn  name4  ttynn  speed

Do you wish to make any changes?  (yes/no)?
```

13. Type *y* if you wish to correct the serial lines and speeds by entering them again. Otherwise, type *n* to move to the next step. The program displays the message:

```
Enter the names of users on each machine:

For machine name1:
  Users on name1:
```

14. Enter a name by typing the login name of a user on the given computer, then press the RETURN key. You may enter more than one name on a line by separating each name with a comma (,) or a space. When you have entered all names for the given computer, press the RETURN key. The program displays the names of the users on the computer and asks if you wish to make corrections:

```
Users on name1 are:

       user1  user2  ...  usern

Do you wish to make any changes?  (yes/no)?
```

15. Type *y* if you wish to correct the user names by entering them again. Otherwise, type *n*. If you type *n*, the program asks for the user names on the next computer.

16. Repeat step 14 and 15 for all remaining computers. When you have given names of users for every computer, the program asks if you wish to enter aliases:

```
Do you wish to enter any aliases? (yes/no)?
```

17. Type *y* if you wish to enter aliases. Otherwise, type *n* to complete the installation. If you type *y*, the program displays the message:

```
Each alias consists of two parts, the first is the alias
name, the second is a list of one or more of the following:
       valid user names
       previously defined aliases
       machine names

  Alias:
```

18. Using the list of aliases you created when you planned the network, type the name of an alias and press the RETURN key. The program displays the message:

```
Users/Aliases:
```

19. If the alias names a single user, type the login name of that user and press the RETURN key. If, on the other hand, the alias names several users, type the login names of the users. If one or more of the users named by the alias are already named by other aliases, type the aliases instead of the login names. If all the users on one computer are named by the alias, type the machine name instead of the login names. In any case, make sure that each item typed on the line is separated from the next by a comma (,) or a space. If there are more items than can fit on the line, type a comma after the last item on that line and press the RETURN key. You can then continue on the next line. After all names and aliases have been typed, press the RETURN key. The program asks for another alias.

20. Repeat steps 18 and 19 for all remaining user aliases in your list. When you have given all aliases, press the RETURN key. The program displays a list of all aliases and their users and asks if you wish to make corrections:

```
Aliases are:

        aliasname:    username  username
        aliasname:    machinename

Do you wish to make any changes?  (yes/no)?
```

21. Type *y* if you wish to enter all aliases again. Otherwise, type *n* to complete the installation.

Once you direct **netutil** to complete the installation, it copies the information you have supplied to the network files, displaying the name of each file as it is updated. Once the files are updated, you may use the **save** option to copy the Micnet files to a floppy disk.

9.3.2 Saving the Micnet Files

You can save copies of the Micnet files on a floppy disk with the **save** option of the **netutil** program. Saving the files allows you to transfer them to the other computers in the network. Before you can save the files, you need to format a floppy disk (see Section 4.2.4 in Chapter 4, "Using File Systems").

To save the files, follow these steps:

1. Log in as the super-user.

2. Type:

   netutil

   Press the RETURN key. The program displays the network utility menu.

3. Type the number 2 and press the RETURN key. The program displays the message:

   ```
   Save to /dev/fdx (yes/no)?
   ```

   where *x* is a drive number.

4. If you wish to use the specified disk drive, insert a blank, formatted floppy disk into the drive, wait for the drive to accept the disk, then type *y* (for "yes") and press the RETURN key. If you do not wish to use the drive, type *n* (for "no") and press the RETURN key. The program displays a prompt asking you for the filename of the disk drive you wish to use. Insert a blank, formatted disk into your chosen drive, wait for the drive to accept the disk, then type the filename of the drive.

In either case, the program copies the Micnet files to the floppy disk.

5. Remove the floppy disk from the drive. Using a soft tip marker (do not use a ball point pen), label the disk "Micnet disk."

As soon as all files have been copied, you can transfer them to all computers in the network.

9.3.3 Restoring Micnet Files

The last step in building a Micnet network is to copy the Micnet files from the Micnet disk to all computers in the network. Do this with the **restore** option of the **netutil** program. For each computer in the network, follow these steps:

1. Log in as the super-user.

2. Type:

   netutil

   Press the RETURN key. The program displays the network utility menu.

3. Type the number 3 and press the RETURN key. The program displays the message:

   ```
   Restore from /dev/fdx (yes/no)?
   ```

   where *x* is the number of a drive.

4. If you wish to use the specified disk drive, insert the Micnet disk into the drive, wait for the drive to accept the disk, then type *y* (for "yes") and press the RETURN key. If you do not wish to use the drive, type *n* (for "no") and press the RETURN key. The program displays a prompt asking you for the filename of the disk drive you wish to use. Insert the Micnet disk into your chosen drive, wait for the drive to accept the disk, then type the filename of the drive.

   In either case, the program copies the network files to the appropriate directories, displaying the name of each file as it is copied. Finally, the program displays the message:

   ```
   Enter the name of this machine:
   ```

5. Type the machine name of the computer you are using and press the RETURN key. The program copies this name to the new */etc/systemid* file for the computer. If necessary, it also disables the serial lines used on the computer, preparing them for use with the network.

When the files have been copied, you may start the network with the **start** option.

9.4 Starting the Network

Once the Micnet files have been transferred to a computer, you can start the network with the **start** option of the **netutil** program. The **start** option starts the Micnet programs that are needed to communicate between the computers in the network.

To start the network, follow these steps for each computer in the network:

1. Log in as the super-user.

2. Type:

    netutil

    Press the RETURN key. The system displays the network utility menu.

3. Type the number 4 and press the RETURN key. The program searches for the */etc/systemid* file. If it finds the file, it starts the network. If not, it asks you to enter the machine name of the computer and then creates the file. The program also asks if you wish to log errors and transmissions. In general, these are not required except when checking or testing the network. When starting the network for the first time, type *n* (for "no") in answer to each question and press the RETURN key.

Once the network has started, you may move to the next computer and start the network there.

Note that, for convenience, you can let each computer start the network automatically whenever the system itself is started. Simply include the command:

    netutil start

in the system initialization file, */etc/rc*, of each computer. To add this command, use a text editor as described in Section 8.11.1 in Chapter 8, "Solving System Problems." You can add the **-x** or **-e** options to this command line if you wish to log transmissions or errors. Even if you do not use these options, Micnet copies a log in and log out message to the system *LOG* file each time you start and stop the network. This means you will need to periodically clear the file. See Section 5.2.9 in Chapter 5, "Maintaining File Systems."

9.5 Testing a Micnet Network

After you have started a network for the first time, you should test the network to see that it is properly installed. In particular, you must determine whether or not each computer is connected to the network.

To test the network, you will need to know how to use the **mail** command (see the XENIX *User's Guide*). The following sections explain how to test the network and how to correct the network if problems are discovered.

9.5.1 Checking the Network Connections

You can make sure that all computers are connected to the network by mailing a short message to **all** (the alias for all users in the network) with the **mail** command. Follow these steps:

1. Choose a computer.

2. Log in as the super-user.

3. Use the **mail** command (see the XENIX *User's Guide*) and the **all** alias to mail the message "micnet test" to all users in the network.

4. Check the mailboxes of each user in the network to see if the message was received. To check the mailboxes, log in as the super-user at each computer and use the **cat** command to display the contents of each user's mailbox. The name of each user's mailbox has the form:

   /usr/spool/mail/login-name

   where *login-name* is the user's login name.

If all users have received the message, the network is properly installed. If the users at one or more computers fail to receive the message, the computers are not properly connected to the network. To fix the problem, you need to locate the computer which has failed to make a connection. The next section explains how to do this.

9.5.2 Using the LOG File to Locate a Problem

You can locate a connection problem by examining the *LOG* files on each computer in the network. The *LOG* files contain a record of the interaction between each pair of computers. There are two *LOG* files for each pair of computers (one file on each computer). The *LOG* files on any given computer are kept in subdirectories of the */usr/spool/micnet* directory. Each subdirectory has as its name the machine name of the other computer in the pair. You can examine the contents of a *LOG* file by typing:

cat /usr/spool/micnet/remote/*machine-name*/LOG

Press the RETURN key. The *machine-name* must be the name of a computer that is paired with the computer you are using.

Each *LOG* file should contain a *startup message* which lists the name of each computer in the pair and the serial line through which the pair is connected. It also shows the date and time at which the network was started. The message should look like:

```
daemon.mn: running as MASTER
Local system: a
Remote system: b, /dev/tty02
Tue Sep 27 22:30:35 1983
```

A startup message is added to the file each time the network starts successfully. If the message is not present, this means that one or more of the network files and directories cannot be found. Make sure that you have used the **restore** option to transfer all the network files to the computer. Also, make sure that the */etc/systemid* file contains the correct machine name for the given computer.

Each *LOG* file will contain a *handshake* message if the connection between the computer pair has been established. The message:

```
first handshake complete
```

is added to the file on a successful connection. If the message is not present, make sure that the network has been started on the other computer in the pair. The network must be started on both computers before any connection can be made. If the network is started on both computers but no handshake message appears, then the serial line may be damaged or improperly connected. Check the serial line to make sure that the cable is firmly seated and attached to the correct RS-232 connectors on both computers. If necessary, replace the cable with one known to work.

If both the startup and handshake messages appear in the *LOG* file but the network is still not working, then there is a problem in transmission. You can create a record of the transmissions and errors encountered while transmitting by restarting the network and requesting Micnet to log all transmissions and errors. Just type *y* (for "yes") when the **start** option asks if you wish to log errors or transmissions.

Error entries contain the error messages that were generated during transmission. Each message lists the cause of the error and the subroutine which detected the error. For example, the message:

```
rsync: bad Probe resp: 68
```

shows that the **rsync** subroutine received a bad response (character 68 hexadecimal) from the other computer. You may use this information to track down the cause of the problem. One common problem occurs when electronic noise passes stray information down the serial line. Make sure that the serial line's cable is properly protected against noise (for example, that the cable does not lie near any electric motor, generator, or other source of electromagnetic radiation). Also, make sure the cable is in good condition.

Transmission entries contain a record of normal transmissions between computers. Each entry lists the direction, byte count, elapsed time, and time of day of the transmission. For example, the entry:

```
rx: 0c 01 22:33:49
```

shows that 12 characters (0c hexadecimal) were received (*rx*) at 22:33:49. The elapsed time for the transmission was 1 second. You can use the records to see if messages are actually being transmitted.

9.5.3 Stopping the Network

You can stop the network with the **stop** option of the **netutil** program. This option stops the Micnet programs, which stops communication between computers in the network.

To stop the network, follow these steps on each computer in the network:

1. Log in as the super-user.

2. Type:

   netutil

   Press the RETURN key. The program displays the network utility menu.

3. Type the number 5 and press the RETURN key. The program stops the network programs running on the computer.

9.5.4 Modifying the Micnet Network

You can modify a Micnet network at any time by changing one or more of the Micnet files. You can reinstall the network with the **netutil** program. For very small changes (for example, correcting the spelling of an alias), you can modify the Micnet files directly with a text editor. The files and their contents are described in detail in Section (M) of the XENIX *Reference Manual*.

Before making any changes to a file, you should make a copy. You can make a copy with the **cp** command. You can replace an old file with the updated file by using the **mv** command. Once one or more files have been changed on one computer, the files must be transferred to the other systems in the network using the **save** and **restore** options. These options can only be used after you have stopped the network.

Note that changes to the *aliases* file will not be incorporated into the system until the **aliashash** program is executed. This program produces the *aliases.hash* file the network needs to resolve aliases. See *aliashash*(M) in the XENIX *Reference Manual* for a description of this command.

9.6 Using a uucp System

You can send and receive mail from other Micnet sites by installing a **uucp** system on one computer in your site. A **uucp** system is a set of XENIX programs that provide communication between computers by using ordinary telephone lines.

To use a **uucp** system with your Micnet network, follow these steps:

1. Choose a computer in the Micnet site.

2. Log in as super-user.

3. Install a **uucp** system on the computer. Installation of a **uucp** system requires a modem and the **uucp** software that is provided with the XENIX *Core System*. See the Appendix C for complete details.

4. Using a text editor, add the entry:

   uucp:*machine-name*

   to the *maliases* file of the computer on which the **uucp** system is installed. The *machine-name* is the name of the computer on which the **uucp** system is installed.

5. On each computer in your site, add the entry:

   uucp:*machine-name*:

   to the *maliases* file. The *machine-name* must be the name of the computer on which the **uucp** system is installed.

You can test the **uucp** system by mailing a short letter to yourself via another site. For example, if you are on the site "chicago", and there is another Micnet site named "seattle" in the system, then the command:

```
mail seattle!chicago!johnd
```

will send mail to the ''seattle'' site, then back to your ''chicago'' site, and finally to the user ''johnd'' in your Micnet network. Note that a **uucp** system usually performs its communication tasks according to a fixed schedule, and may not return mail immediately.

Chapter 10

Installing Device Drivers

10.1 Introduction

This chapter explains how to install device drivers so you can attach additional peripheral devices to your system. It describes how configuration files are used to define your system and how to use the XENIX facilities to install new device drivers.

10.1.1 What is a XENIX Device Driver?

For each peripheral device in a XENIX system, there must be a *device driver* to provide the software interface between the device and the system. A XENIX device driver is a set of routines that communicates with a hardware device to control and perform input/output (I/O) operations.

10.1.2 Why Install a Device Driver?

Before a hardware device can be used in the XENIX operating system, the device driver associated with it must be installed in the software system. A device driver is normally supplied as a single software module or file. Installing this software module is as essential to the operation of the device as the actual hardware installation. It must be completed before the device can be used.

10.1.3 Before You Start

Before you install a device driver on your XENIX system, you should:

- Install the hardware device on your computer system according to the manufacturer's instructions.

- Know how to edit data files using one of the XENIX text editors. Refer to the XENIX *Text Processing Guide* for information.

- Be familiar with the format and content of the system configuration file */lib/sys/config.sys* that is provided with your XENIX system.

The exact procedure used to install a new device driver will vary for each type of device. The examples presented in this chapter are intended to serve as guidelines for installing device drivers. When installing an actual device driver in your system, you should supplement the specific installation instructions that are provided with the device driver software with the instructions given here.

10.1.4 What You Need

To adequately illustrate the process of installing device drivers, we have selected two devices that you are likely to add to your standard system configuration. We assume, for this example, that your system is already in use and that you have recently acquired a new high quality printer and a new high capacity disk drive, each of which you wish to install on your system. With each of these devices, a single floppy disk has been supplied that contains the necessary software to enable you to complete the installation.

We also assume, for this example, that you have installed the hardware devices on your system according to the manufacturer's instructions and that you are now ready to install the software so you can use the new devices.

10.2 Overview of the Installation Procedure

The */lib/sys* directory has been reserved for installable device driver software modules and their associated files. The system configuration file, *config.s*, is also located in this directory. You must add a special device file to the */dev* directory for each of the new devices in order to give other programs in the system access to the devices. Most of the steps you will follow to install the new device drivers involve adding new files and modifying existing files in these directories.

The process of installing new device drivers in your system consists of five major steps.

1. Install the device driver software modules in the system.
2. Modify the system configuration file to include the new drivers.
3. Create special device files for each new device.
4. Load the newly installed device drivers in the system.
5. Create a new version of the XENIX system which includes the new drivers.

Each of these steps are described fully in the following sections.

10.2.1 Installing the Device Driver Modules

To begin the installation of the new device drivers, you will usually copy the device driver software modules from the disk that is supplied with the device to the */lib/sys* directory. However, if the instructions provided by the manufacturer of the device conflict with the instructions presented here, the manufacturer's instructions take precedence.

Continuing with the example illustration, follow the instructions given below to install the device driver modules:

1. Check the contents of the floppy disks provided with the devices. To do this, insert each disk, in turn, in drive 0 and type:

   ```
   tar tf /dev/rfd0
   ```

   The disk provided with the high quality printer will typically display the following contents:

   ```
   /lib/sys/config.sys
   /lib/sys/hqp.x
   ```

   The disk provided with the high capacity disk drive will typically display the following contents:

   ```
   /lib/sys/config.sys
   /lib/sys/hcd.x
   ```

   Remember that the new devices and their associated device driver software used in this example of an installation procedure are theoretical; therefore, the system responses to the instructions are also theoretical.

2. Check the current contents of the */lib/sys* directory to make sure you will not overwrite an existing file by installing the device driver files. Typing the command line:

   ```
   ls /lib/sys
   ```

   will display the contents of the directory. If there is an existing file of the same name as one you wish to install, you must rename the existing file in order to preserve its contents. In our example, we see a name clash with the *config.sys* file. A *config.sys* file appears on each disk and the default *config.sys* file appears in the */lib/sys* directory.

3. Rename the existing *config.sys* file in the */lib/sys* directory to *Mconfig.sys* to preserve its contents when the device driver files are installed. To do this, type:

   ```
   cd /lib/sys
   mv config.sys Mconfig.sys
   ```

4. Insert the disk that contains the device driver software for the printer in drive 0. Copy the files from this disk on to the system by typing:

   ```
   tar xf /dev/rfd0
   ```

5. Rename the printer's *config.sys* file to *config.hqp* to preserve its contents. To do this, type:

   mv config.sys config.hqp

6. Insert the disk that contains the device driver software for the disk drive in drive 0. Copy the files from this disk on to the system by typing:

   tar xf /dev/rfd0

7. Rename the disk drive's *config.sys* file to *config.hcd* to preserve its contents. To do this, type:

   mv config.sys config.hcd

8. Restore the system's original *config.sys* file to its original name by typing the command line:

   mv Mconfig.sys config.sys

You are now ready to modify the system configuration file to include the new device drivers.

10.2.2 Modifying the System Configuration File

The next step in installing the device drivers is to edit the configuration file, */lib/sys/config.sys*, to include the new devices and their device drivers. The format of this file is described in full in the XENIX *Programmer's Guide*. The device specifications you need to include in the *config.sys* file should be supplied to you with the installation instructions for the device.

Before you begin editing the *config.sys* file, you should have a general understanding of how device drivers are specified in this file. The lines in the *config.sys* file are either of two types: a comment line, which begins with a (#) character and allows you to include some descriptive information, or an installable device driver specification, which begins with the characters "DD". Each device driver specification line contains an additional seven fields. These fields contain the following information:

- The *name* of the device. This is a mnemonic name associated with the device that is being installed. It is important that this name be used exactly as specified in the installation instructions, since the device driver software uses this name.

- The *command* which specifies whether the device you are installing will be added to the current configuration, will replace a previously existing device, or will share a device interrupt level with another device. Interrupt levels are described further below.

- The *major device number* of this device, if it is a block device, or a –1, if it is a character device only.

- The *major device number* of this device, if it is a character device, or a –1, if it is a block device only.

- A *reserved field* which should always contain an asterisk (*) character.

- The list of *interrupt vector numbers* that are associated with this device. This list is separated by commas and contains numbers in the range zero to seventy one. If the device is a non-interrupting type, this field should contain a –1.

- The *full pathname* of the file that contains the device driver module.

In most cases, you simply copy the device driver specification line from the *config.sys* file that is supplied with the device into the *config.sys* file that describes your system. You must be careful to check for clashes between existing devices and new devices. These clashes can occur in any of three fields in a device driver specification, the major device number for either the block or character device fields, and the interrupt vector numbers used by the device.

The rules for resolving these clashes are fairly simple:

- If an existing device has the same major device number as that given for the new device, choose a new, unique number for the new device and use that as the major device number. This rule applies to both the block and character device fields. Note that if you have to change the major device number from that supplied in your installation instructions, you must use this new number when you create the special device files described in Section 10.2.3. If you wish to replace an existing device with the new device, you should choose the same major device number as the existing device and specify the **REP** option in the command field.

- If the interrupt vector number for the new device is different than all others listed in the existing *config.sys* file, put the string "ADD" in the device specification line in the command field and set the interrupt vector field as shown in the supplied instructions.

- If the interrupt vector number is the same as in an existing device specification, put the string "SHR" in both the existing and new device specification descriptions in the command field. The interrupt vector numbers should be the same for both devices.

Follow the instructions below to modify the *config.sys* file to include the new printer and disk drive:

1. Examine the contents of the printer configuration file *config.hqp* and the disk drive configuration file *config.hcd*. To do this, type:

   cat config.hqp config.hcd

   For purposes of continuing with our example, we will assume the contents of *config.hqp* to be:

```
#Whizzo Computer Products High Quality Printer Driver
#
DD      hqp     ADD     -1      7 * 5           /lib/sys/hqp.x
```

   For purposes of continuing with our example, we will assume the contents of *config.hcd* to be:

```
#Whizzo Computer Products High Capacity Disk Drive Driver
#
DD      hcd     ADD     2       2 * 1           /lib/sys/hcd.x
```

2. Examine the contents of the system's *config.sys* file by typing the command line:

   cat config.sys

   A standard *config.sys* file for a XENIX-286 computer that installs the drivers for a floppy disk drive, a serial interface, and a parallel interface consists of the following:

```
#       Example config.sys file
#
DD      fd      ADD     2       2 * 6           /lib/sys/fd.x
DD      sa      ADD     -1      5 * 3,4         /lib/sys/sa.x
DD      pa      ADD     -1      6 * 5,7         /lib/sys/pa.x
```

3. Resolve the major device number conflict between the new *hcd* driver and the existing *fd* driver by changing both major device numbers for the *hcd* device to one. The corrected device driver specification for *hcd* will be:

```
DD      hcd     ADD     1       1 * 1           /lib/sys/hcd.x
```

4. Resolve the interrupt level number conflict between the new *hqp* driver and the existing *pa* driver by specifying the "SHR" string so that both drivers can share interrupt level five. The corrected device driver specifications for the drivers will be:

```
DD      pa      SHR     -1      6 * 5,7         /lib/sys/pa.x
DD      hqp     SHR     -1      7 * 5           /lib/sys/hqp.x
```

5. To conclude, the modified system configuration file *config.sys* that incorporates the new drivers should look like:

```
#       config.sys file with Whizzo devices added
#
DD      fd      ADD     2       2 * 6           /lib/sys/fd.x
DD      sa      ADD     -1      5 * 3,4         /lib/sys/sa.x
DD      pa      SHR     -1      6 * 5,7         /lib/sys/pa.x
DD      hqp     SHR     -1      7 * 5           /lib/sys/hqp.x
DD      hcd     ADD     1       1 * 1           /lib/sys/hcd.x
```

You have completed the process of modifying the system configuration file to include the new devices.

10.2.3 Creating Special Device Files

A file must exist in the file system for each newly installed device so that programs can access the new devices. These files, called *special device files*, are usually located in the */dev* directory. The installation instructions supplied with the device will specify the name you should use for the special device file and the other parameters associated with it.

Special device files are created using the **mknod** command with a set of required parameters. You must supply the name of the special device file, the device type (block or character), and the major and minor device numbers associated with the device.

To create special device files for the two new devices in our example, follow the instructions below:

1. Move to the */dev* directory where you will create the special device files. To do this, type:

   cd /dev

2. Create the special device file for the disk drive as a block device. To do this, type the command line:

   /etc/mknod hcd0 b 1 0

where *hcd0* is the name of the special device file, *b* specifies that the device is a block device, *1* is the major device number, and *0* is the minor device number. Note that we used the modified device numbers for the *hcd* driver rather than the ones originally supplied on the disk.

3. Create the special device file for the disk drive as a character device. To do this type the command line:

   ```
   /etc/mknod rhcd0 c 1 0
   ```

   Note that an "r" has been prepended to the special device file name to indicate that this is the character special ("raw") form of the device.

4. Create the special device file for the high speed printer. This device is only a character device, so only one special device file needs to be created for it. To create the file, type the command line:

   ```
   /etc/mknod hqp c 7 0
   ```

You have finished creating the necessary special device files for your two new devices.

10.2.4 Loading the Device Drivers

This section describes how to load the newly installed device driver modules so they become part of the XENIX system. Once your device drivers have been correctly installed, the information in this section is not relevant to the ongoing operation of the system.

To begin the loading procedure, you must boot (restart) the system. When you boot the system, the screen display looks like this:

```
SYSV   XENIX Boot

Enter: hd program
              fd program
              dos
              cf [-c config_file] [device program]

Press Enter for default:    hd /xenix
:
```

To add the new device drivers to the system, you must respond to the system prompt ":" with some form of the **cf** command. This command instructs the **boot** program to examine the *config.sys* file for changes and causes any newly installed device drivers to be incorporated into the system. The most commonly used form of the **cf** command is simply:

```
cf
```

This command automatically uses the */lib/sys/config.sys* file as the system configuration file and the */lib/sys/xenix* file as the operating system image. You can specify any operating system that you want to use. For example, typing:

```
cf hd /xenix.old
```

will link the installable device drivers to the operating system image specified by */xenix.old*. The *device* parameter may be either *hd* (hard disk) or *fd* (floppy disk). The *program* parameter should specify the complete pathname of whatever file is to be used as the operating system image (default is */lib/sys/xenix*).

You may also invoke the **cf** command with the **–c** option which allows you to specify a system configuration file other than the default file */lib/sys/config.sys*. For example, typing:

```
cf –c /conf
```

will use the file */conf* as the system configuration file and */lib/sys/xenix* as the operating system image when linking the new device drivers to the system.

In order to incorporate newly installed device drivers into the system, you must invoke the **cf** command during the boot process. The functions performed by the **cf** command cause the boot process to take much longer than usual. Normally, you will have to do this once, and subsequent boot processes will take a short time to complete.

10.2.5 Creating a New XENIX Kernel

In order to create a new default version of the operating system that incorporates the newly installed device drivers, you can use the **xconf** utility. Normally, this program will be automatically executed during system initialization by the */etc/rc* shell script. If the configuration of the system includes newly installed device drivers, */etc/rc* will build a new operating system image and place this in the file */xenix*. The existing */xenix* will be moved to the file */xenix–*.

If you wish to prepare several different operating system configurations, invoke **xconf** directly. For example, typing the command line:

```
xconf –o newxenix –c newconfig hd /lib/sys/xenix
```

will build a new operating system image in the file *newxenix*, using the configuration file *newconfig* and the partial operating system image */lib/sys/xenix*.

Note that you should create the new operating system image with the name */xenix*. If you do not do this, neither the **ps** or **pstat** commands will produce correct output when you execute them, unless you use the "**–n**" option.

10.2.6 The installidd Utility Program

Your version of XENIX has a utility, **installidd**, that makes this installation process much simpler. The **installidd** utility automatically handles the resolution of major device number and interrupt vectors. We recommend that you use the **installidd** utility instead of the steps outlined earlier. See **installidd**(M) for more information.

Appendix A
XENIX Special Device Files

A.1 Introduction

This appendix contains information needed to create file systems and add terminals to the XENIX system. For a full description of the special files mentioned here, see Sections C and M.

A.2 File System Requirements

Many of the file system maintenance tasks described in this guide require the use of special filenames, block sizes, and gap and block numbers. The following sections describe each in detail.

A.3 Special Filenames

A special filename is the name of the device file (special block or character I/O) corresponding to a peripheral device, such as a fixed or floppy disk drive. These names are required in such commands as **mkfs**, **mount**, and **df** to specify the device containing the file system to be created, mounted, or searched.

A.3.1 Floppy Diskette Special Filenames

The following tables lists the special filenames and corresponding formats for the floppy diskette drives:

Drive 0 / Drive A:

Special Filename	Format
/dev/fd048	9 sectors, double sided
/dev/fd048ss8	8 sectors, single-sided
/dev/fd048ds8	8 sectors, double-sided
/dev/fd048ss9	9 sectors, single-sided
/dev/fd048ds9	9 sectors, double-sided
/dev/fd096ds15	15 sectors, double-sided
/dev/fd0	15 sectors, double-sided
/dev/fd096	15 sectors, double-sided

Drive 1 / Drive B:

Special Filename	Format
/dev/fd148	9 sectors, double-sided
/dev/fd148ss8	8 sectors, single-sided
/dev/fd148ds8	8 sectors, double-sided
/dev/fd148ds9	9 sectors, single-sided
/dev/fd148ds9	9 sectors, double-sided
/dev/fd196ds15	15 sectors, double-sided
/devfd196	15 sectors, double-sided

A.3.2 Fixed Disk Special Filenames

The following tables list the special filenames and corresponding partition number for the fixed disks:

Fixed Disk 1:

Special Filename	Format
/dev/root	User-defined *system* file system
/dev/usr	User-defined *user* file system
/dev/rhd00	Raw Device (Entire Disk)
/dev/hd00	Block Device (Entire Disk)
/dev/rhd01	Raw Partition 1
/dev/hd01	Partition 1
/dev/rhd02	Raw Partition 2
/dev/hd02	Partition 2
/dev/rhd03	Raw Partition 3
/dev/hd03	Partition 3
/dev/rhd04	Raw Partition 4
/dev/hd04	Partition 4
/dev/hd0a	Active Partition
/dev/rhd0a	Raw Active Partition
/dev/hd0d	DOS Partition
/dev/rhd0d	Raw DOS Partition

Fixed Disk 2:

Special Filename	Format
/dev/rhd10	Raw Device (Entire Disk)
/dev/hd10	Block Device (Entire Disk)
/dev/rhd11	Raw Partition 1
/dev/hd11	Partition 1
/dev/rhd12	Raw Partition 2
/dev/hd12	Partition 2
/dev/rhd13	Raw Partition 3
/dev/hd13	Partition 3
/dev/rhd14	Raw Partition 4
/dev/hd14	Partition 4
/dev/hd0a	Active Partition
/dev/rhd0a	Raw Active Partition
/dev/hd0d	DOS Partition
/dev/rhd0d	Raw DOS Partition

Use */dev/usr* and */dev/root* whenever possible. The raw device names are used with the **badtrack** and **fdisk** commands.

A.3.3 Tape Disk Special Filenames

The following table lists the special filenames for the tape drive system:

Special Filename	Format
/dev/mt0	Block Device, rewinds after use
/dev/cmt0	Continuous Block Device
/dev/rmt0	Raw Device, rewinds after use
/dev/rcmt0	Continuous Raw Device

A.3.4 Cartridge Disk Special Filenames

The following table lists the special filenames for the cartridge disk system:

Cartridge Disk 0:

Special Filename	Format
/dev/cd0	File System
/dev/rcd0	Raw Device File System
/dev/cdbt0	Boot Track
/dev/rcdbt0	Raw Device Boot Track

Cartridge Disk 1:

Special Filename	Format
/dev/cd1	File System
/dev/rcd1	Raw Device File System
/dev/cdbt1	Boot Track
/dev/rcdbt1	Raw Device Boot Track

A.4 Block Sizes

The block size of a disk is the number of blocks of storage space available on the disk. (A block is typically 1024 bytes of storage) However, some commands, such as **tar** use half-block sizes (512 bytes). Some commands require you to enter blocks or display sizes as half-blocks.

The following table lists the block sizes of floppy diskettes for your computer:

Diskette Format	Block Size
8 sectors, single-sided	160
8 sectors, double-sided	320
9 sectors, single-sided	180
9 sectors, double-sided	360
15 sectors, double-sided	1200

A standard 20 megabyte fixed disk has 20,000 full blocks divided into at least 4 partitions. The number and size of each partition depend on the configuration set during installation.

A.5 Gap and Block Numbers

The **mkfs** command uses gap and block numbers to describe how the blocks are to be arranged on a disk. The following table lists the gap and block numbers for the floppy diskettes and fixed disk used with your computer:

Device	Gap	Blocks
Diskettes		
9 sectors per track	3	18
15 sectors per track	3	30
Fixed Disk	1	34

A.6 Terminal and Network Requirements

The **enable** and **disable** commands (used to add and remove terminals on a system) and the install option of the **netutil** program (used to build a network) require the names of the serial lines through which a terminal or network is connected. Besides the console, XENIX has several special filenames devoted to the optional serial lines of your computer. You must install the serial communications hardware to access these files. The following table lists the device special filename of the serial lines on a typical configuration:

Filename	**Line**
/dev/console	Serial line for system console
/dev/tty00	Serial line 0
/dev/tty01*	Serial line 1
/dev/tty02	Serial line 2
/dev/tty03	Serial line 3
/dev/tty04	Serial line 4
/dev/tty05	Serial line 5
/dev/tty06	Serial line 6
/dev/tty07	Serial line 7
/dev/tty08	Serial line 8
/dev/tty09	Serial line 9

* *tty01* can only be used if you reconfigure the system.

Appendix B
XENIX Directories

B.1 Introduction

This appendix lists the most frequently used files and directories in the XENIX system. Many of these files and directories are required for proper XENIX operation and must not be removed or modified. The following sections briefly describe each directory.

B.2 The Root Directory

The root directory (/) contains the following system directories:

/bin	XENIX command directory
/dev	Device special directory
/etc	Additional program and data file directory
/lib	C program library directory
/mnt	Mount directory (reserved for mounted file systems)
/usr	User home directories
/tmp	Temporary directory (reserved for temporary files created by programs)

All of the above directories are required for system operation.

The root directory also contains a few ordinary files. Of these files, the most notable is the *xenix* file which contains the XENIX kernel image.

B.3 The /bin Directory

The */bin* directory contains the most common XENIX commands, that is, the commands likely to be used by anyone on the system. The following is a list of a few of the commands:

basename	**echo**	**passwd**	**su**
cp	**expr**	**rm**	**sync**
date	**fsck**	**sh**	**tar**
backup	**login**	**sleep**	**restore**
dumpdir	**mv**	**stty**	**test**

These commands and all others in the */bin* directory are required for the XENIX operating system and must not be removed.

B.4 The /dev Directory

The */dev* directory contains special device files that control access to peripheral devices. All files in this directory are required and must not be removed. The following is a list of the files:

/dev/console	System console
/dev/lp	Lineprinter
/dev/mem	Physical memory
/dev/null	Null device (used to redirect unwanted output)
/dev/rXX	Unbuffered interface to corresponding device name
/dev/root	Root file structure
/dev/swap	Swap area
/dev/ttyXX	Terminals
/dev/tty	The terminal you are using

B.5 The /etc Directory

The */etc* directory contains miscellaneous system program and data files. All files are required, but many may be modified.

The following program and data files must not be removed or modified.

/etc/mtab	Mounted device table
/etc/mount	For mounting a file structure
/etc/mkfs	For creating a file structure
/etc/init	First process after boot

The following data files may be modified, if desired. No file may be removed.

/etc/passwd	Password file
/etc/rc	Bootup shell script
/etc/ttys	Terminal set up
/etc/termcap	Terminal capability map
/etc/motd	Message of the day
/etc/profile	Profile file

B.6 The /lib Directory

The */lib* directory contains runtime library files for C and other language programs. The directory is required and must not be removed.

B.7 The /mnt Directory

The */mnt* directory is an empty directory reserved for mounting removable file systems.

B.8 The /tmp Directory

The */tmp* directory contains temporary files created by XENIX programs. The files are normally present when the corresponding program is running, but may also be left in the directory if the program is prematurely stopped. You may remove any temporary file that does not belong to a running program.

B.9 The /usr Directory

The */usr* directory contains the home directories of all users on the system. It also contains several other directories that provide additional XENIX commands and data files.

The */usr/bin* directory contains more XENIX commands. These commands are less frequently used or considered nonessential to XENIX system operation.

The */usr/include* directory contains header files for compiling C programs.

The */usr/lib* directory contains more libraries and data files used by various XENIX commands.

The */usr/spool* directory contains various directories for storing files to be printed, mailed, or passed through networks.

The */usr/tmp* directory contains more temporary files.

The */usr/adm* directory contains data files associated with system administration and accounting. In particular, the */usr/adm/messages* file contains a record of all error messages sent to the system console. This file is especially useful for locating hardware problems. For example, an unusual number of disk errors on a drive indicates a defective or misaligned drive. Since messages in the file can accumulate rapidly, the file must be deleted periodically.

B.10 Log Files

A variety of directories contain log files that grow in size during the normal course of system operation. Many of these files must be periodically cleared to prevent them from taking up valuable disk space (see Section 5.2.9 in Chapter 5, "Maintaining File Systems"). The following table lists the files (by full pathname) and their contents:

Filename	Description
/etc/ddate	Records date of each backup.
/usr/adm/pacct	Records accounting information; grows rapidly when process accounting is on.
/usr/adm/messages	Records error messages generated by the system when started.
/usr/adm/wtmp	Records user logins and logouts.
/usr/adm/sulog	Records each use of the **su** command; grows only if the option is set in the */etc/default/su* file.
/usr/spool/at/past	Records each use of the **at** command.
/usr/spool/micnet//LOG*	Records transmissions between machines in a Micnet network. The * must be the name of a remote machine connected to the current machine.

Appendix C
Building a Communication System

C.1 Introduction

This appendix explains how to build a communication system for your computer using a normal telephone line and a Hayes Smartmodem 1200. A communication system provides a way to:

- Log in to the computer from a remote terminal or computer.
- Use the **cu** command to call and log in to other computers.
- Use the **uucp** command to copy files to and from remote computers.
- Use the **uux** command to execute a remote mail program (**rmail**) on a remote computer.

In other words, the communication system is intended to give access to terminals and computers that cannot be connected to your computer through a direct serial line. In particular, the communication system is a practical solution to the problem of two Micnet networks (see the XENIX *Operations Guide*) that cannot be connected because of distance or cost of cable.

All communication tasks are supported by a variety of files and directories. In addition, the tasks invoked by the **uucp** and **uux** commands are actually performed by a system of underlying programs, called the **uucp** system. The files and underlying programs are described later in this appendix.

The following sections explain how to install the modem and prepare the programs you need to build a communication system. They also explain how to install and maintain a **uucp** system. Many of the administrative tasks associated with installing and setting up a **uucp** system are handled by the **uuinstall** utility. The next section explains how this utility is used. The last section of this appendix presents the program listing of the **dial** program used to communicate with the modem. This code may be used to create a modified **dial** program that communicates at a different baud rate (e.g., 300 baud) or with a different modem.

C.2 Setting Up a Uucp System

The **uuinstall** program is used to manage the information contained in the control files for the **uucp** system. There are five of these files:

/etc/systemid	Contains the name by which your system is known on the **uucp** network.
/usr/lib/uucp/L.sys	Contains a description of other systems you can call, or which call you, on the **uucp** network.

/usr/lib/uucp/USERFILE	Contains a description of the access you allow to other systems and users on those systems, and to the files of the local system.
/usr/lib/uucp/L-devices	Contains a description of the devices that are used to connect to other systems on the **uucp** network.
/usr/lib/uucp/L-dialcodes	Contains a list of abbreviations that are used in the dialing codes for placing calls to other systems.

To invoke the **uuinstall** utility, type "uuinstall" on the command line. The screen displays the main menu shown below:

```
                  UUCP Administration Utility


     Choose one of the following options:
            1. Examine or update system identification.
            2. Examine or update dial-in or dial-out devices
            3. Examine or update dialing code abbreviations.
            4. Examine or update system connections.
            5. Examine or update UUCP users.
            6. Terminate this program.

     Choose an option:
```

The **uuinstall** utility will keep returning to this display after performing the action that you request. When you have completed all the required changes to the **uucp** control files, you should enter option '6'. The **uuinstall** program will display:

```
     Do you want to update the UUCP control files?
```

This gives you one last chance to decide whether you wish to make the changes to the control files that you have indicated. Typing "y" after this prompt will cause the control files to be updated. Any other response will cause the **uuinstall** program to terminate without making any changes to the files. Each of the options listed in the main **uuinstall** menu are described in detail later in this appendix.

You can also invoke the **uuinstall** program with a **–r** command line option. This allows you to read the current settings of the **uuinstall** menu options, but will not allow you to make any changes. In order to run **uuinstall**, you must be the super-user. Refer to **uuinstall**(C) in the XENIX *Reference Manual* for more information.

C.3 What You Need

To install a communication system on your computer, you will need:

- A Hayes Smartmodem 1200.
- A standard telephone jack for access to the telephone system (touch tone line required).
- An RS-232 serial line (or serial port) on your computer.
- An RS-232 cable to connect the serial line to the modem.

For proper operation of the modem, the RS-232 cable must provide the pin connections shown below. Note that the computer's serial connector must have a DTE (Data Terminal Equipment) configuration. The modem is assumed to have a DCE (Data Communications Equipment) configuration.

Pin Connections

Computer (DTE)	Modem (DCE)
1	1
2	2
3	3
6	6
7	7
8	8
20	20

These pin connections are explained in the *Hayes Smartmodem 1200 Reference Manual*. (This manual will be referred to as the *Hayes Reference Manual* in this appendix.).

Make sure you inform the telephone company of your intent to use a modem with your telephone line. See the *Hayes Reference Manual* for details.

Finally, since many of the tasks you must perform require special permissions, you must log in to your computer's super-user account before performing them. Check with your computer's system manager before proceeding with this installation, or turn to the XENIX *Operations Guide* for instructions on how to log in as the super-user.

C.4 Installing the Modem

Installing the modem is the next step in creating a complete communication system. The installation has four steps:

1. Choose a serial line.
2. Set the dialing configuration.
3. Connect the modem.
4. Test the connection.

The following sections explain each step in detail.

C.4.1 Choose a Serial Line

You must choose the RS-232 serial line you wish to use with the system and connect it to the modem. If there are no lines available, you must install a new serial line or make one available by removing any device connected to it. If you remove a terminal, make sure no one is logged in.

Once you have chosen a serial line, find the name of the device special file associated with the line by looking in Appendix A of the XENIX *Operations Guide*. The filename should have the form:

/dev/tty*nn*

where *nn* is the number of the corresponding line. For example, */dev/tty00* usually corresponds to serial line 0. You need the filename for later steps.

C.4.2 Set the Dialing Configuration

In this communication system, your modem can be used to both send and receive calls. You must set the appropriate switches on the modem. Follow these steps:

1. Remove the front cover of the modem and locate the 8-pin configuration switch. (See the *Hayes Reference Manual* for instructions on how to remove the cover and the locate the switch.)
2. Set the pins on the configuration switch to the following positions:

pins	1	2	3	4	5	6	7	8
positions	up	up	down	down	up	up	up	down

3. Replace the front cover.

C.4.3 Connect the Modem

Once your modem's dialing configuration is set, you are ready to connect the modem to your computer. Review the installation instructions given in the *Hayes Reference Manual*, then follow these steps:

1. Connect the RS-232 serial cable to the serial line connector on the modem, then to the serial line connector on your computer. Make sure the cable is fully connected.

2. Plug the telephone line cable into the telephone connector on the modem, then into the telephone wall jack.

3. Plug in the power cord of the modem.

C.4.4 Test the Modem

As the last step of the modem installation, you should test the modem to make sure that it can send and receive calls. Once you have verified that the modem is working, you can begin to use the communication system.

To test the modem, follow these steps:

1. Start the computer and log in as the super-user.

2. Disable the modem's serial line by typing:

   disable /dev/tty*nn*

   where *nn* is the serial line's number.

3. Turn on the modem's power.

4. Make sure the volume switch on the modem is at an appropriate level. You must be able to hear the modem to carry out this test successfully. See the *Hayes Reference Manual* for the location of this switch.

5. Invoke the **dial** program using a command line of the form:

   /usr/lib/uucp/dial /dev/tty*nn* *number speed*

   where */dev/ttynn* is the filename of your serial line, and *number* is your telephone number (the number of the telephone jack your modem is connected to). For example, if your serial line is */dev/tty00* and your telephone number is "5551234", type:

/usr/lib/uucp/dial /dev/tty00 5551234 1200

6. Listen carefully to the modem. You should hear each digit as the number is dialed; then you should hear the busy signal when the telephone system tries to make connection with your modem.

7. If you hear the busy signal, wait a few moments and listen carefully for the modem to hang up. The modem automatically discontinues any call it cannot connect.

8. If you don't hear the busy signal, make sure you have connected the modem to the telephone jack. Make sure the jack is connected to the phone system, and make sure you gave the correct number when invoking **dial**.

9. If you did not hear the modem dial, make sure the volume switch is up. See that the modem is connected to the correct serial line and that the cable connection is tight. Make sure you gave the correct filename when invoking **dial**, and verify that the modem's power is on.

C.5 Creating a Dial-in Line

You can create a dial-in line for remote terminals or computers by enabling the modem's serial line with the **enable** command after making sure your modem's serial line has an appropriate */etc/ttys* file entry. Once the line is enabled, any user at a remote terminal or computer can log in to your computer by calling your modem and following the ordinary login procedure. To create a dial-in line, follow these steps:

1. Log in as the super-user.

2. Use the **cat** command to examine the contents of the */etc/ttys* file.

3. Locate the entry in this file that corresponds to your serial line. The correct entry contains the name of your serial line and must have the form:

   03tty*nn*

   where *nn* is the number of your serial line.

4. If necessary, use a XENIX text editor to change the entry or create a new entry for your serial line. Make sure the first two digits in the entry are "0" and "3", respectively.

5. Save the edited file and exit the editor.

6. Type:

enable /dev/tty*nn*

where *nn* is the number of your serial line. This enables the line for logins.

Your computer will now receive calls from remote terminals or computers, and it will prompt for a login name.

C.6 Creating a Dial-out Line

You can create a dial-out line by disabling the modem's serial line, creating the call unit files, and finally setting up the */usr/lib/uucp/L-devices* file. A dial-out line lets you call and log in to other computers by using the **cu** command. The **cu** command uses the */usr/lib/uucp/L-devices* file to locate the modem's serial line and set the proper line speed when these values are not given explicitly in the **cu** command line. The call unit files are actually specially named files that are linked to your modem's serial line and used by **uucp** system programs.

The following sections explain how to create the necessary files and enable your line.

Note

Your modem's serial line cannot be both dial-in and dial-out at the same time. However, you can alternate between dial-in and dial-out at different times of the day by enabling or disabling the serial line as needed. Make sure you wait at least one minute between each invocation of the **enable** and **disable** commands.

C.6.1 Create the Call Unit Files

You must create two new device files, called "call unit" files, using the **ln** command and your serial line file. Follow these steps:

1. Log in as the super-user.

2. Check for any existing call unit files by using the **l** command. Type:

   l /dev/cu*

   and examine the output. Call unit filenames have the form:

/dev/cua*n*

and:

/dev/cul*n*

where *n* is the same number as the corresponding serial line. If these files exist, you can skip the next step and continue with setting up the */usr/lib/uucp/L-devices* file.

3. Use the **ln** command to create the call unit files. For example, if your serial line is named */dev/tty00*, type the commands:

   ln /dev/tty00 /dev/cua0

   and:

   ln /dev/tty00 /dev/cul0

4. Use the **chmod** command to change the access mode of the call unit files to read and write for everyone. For example, the command:

   chmod ugo+rw /dev/cua0 /dev/cul0

   sets the appropriate permissions for the */dev/cua0* and */dev/cul0* files.

C.6.2 Set Up the L-devices File

The */usr/lib/uucp/L-devices* file defines the devices you intend to use to implement the dial-out line. The file is also used by programs in the **uucp** system (as described later). The file contains one or more entries of the form:

type line call-unit speed

where *type* must be "ACU" if you are using an automatic call unit (modem) or "DIR" if you are using a direct serial line, *line* and *call-unit* are the */dev/cul* and */dev/cua* call unit filenames, respectively, and *speed* is the line speed or baud rate for transmissions. The call unit files are assumed to be in the */dev* directory, so the full pathname is not required. For example, the following entry:

ACU cul0 cua0 1200

defines the */dev/cul0* device as the line, */dev/cua0* as the call-unit, and 1200 as the line speed.

With the Hayes modem, the *speed* must be set to 1200. Note that if you adapt the communication system for other purposes and give a *line* to a hardwired device (e.g., a direct serial line to another computer), you must use the number 0 for the *call-unit* field instead of a device name.

Use the **uuinstall** program to set up this file. (Refer to Section B.2.) When you invoke the **uuinstall** program, select option 2 when the main menu is displayed. You will then be asked if you wish to see the current devices. Type "y" and a screen similar to the following one will appear:

```
#      Type    Line     Call-Unit      Speed

0.     ACU     cul0        cua0        1200
1.     DIR     tty00                   1200
```

This display corresponds to the two lines in the L-devices file which would be:

ACU cul0 cua0 1200
DIR tty00 0 1200

The program now asks whether you wish to add or delete an entry in the file. Entries are always deleted by specifying the entry number (#) shown in the first column on the screen display. If you request that an entry be added, you will be prompted for the type of the unit, which can be an ACU or a direct line (respond "a" for ACU or "d" for a direct line). If you simply press ENTER in response to this prompt, the **uuinstall** display will return to the main menu.

If you specify an ACU, you will then be prompted for the unit number of the calling unit and the line. Respond with just the number in each case; the **uuinstall** program will generate the appropriate "cua" and "cul" strings. If you specify a direct line, you will be prompted for the line number. Finally, you will be prompted for the speed of the line. Your response is checked and, if it is invalid, the program prompts you for a valid response.

C.6.3 Enable the Serial Line

Enabling the serial line is the last step in creating a dial-out line. To enable the serial line, follow these steps:

1. Make sure your modem has been installed and tested.

2. Make sure you are logged in as the super-user.

3. Disable the modem's serial line by typing:

   disable /dev/tty*nn*

where *nn* is the number of your modem's serial line. If the line is already disabled, the command displays an error message that you can safely ignore.

You will now be able to call other computers that have dial-in lines by using the **cu** command. For a complete description of the command, see **cu**(C) in the XENIX *Reference Manual.*

C.7 Installing a Uucp System

A **uucp** system is a set of files and programs that let you use the **uucp** and **uux** commands to transfer files and commands between computers connected by your communication system. Before you can use the **uucp** and **uux** commands, you must install the **uucp** system by modifying a number of **uucp** system files using the **uuinstall** utility program.

The **uucp** system actually provides two different methods of interaction with other computers. One method requires a dial-in line through which remote computers can log in and transfer files and commands. With this method, your computer is called a "dial-in site." The other method requires a dial-out line through which your computer can call other computers. With this method, your computer is called a "dial-out site." Each method requires its own set of **uucp** system files.

Although you can install files for both methods, you can only use one method at a time. It is possible, however, to use alternate methods at different times of the day by creating a shell script that automatically enables or disables the line to permit dialing in or dialing out.

The following sections explain how to set up the files for both methods of interaction. They also explain how to create a transmission schedule and develop a **cron** script to implement the schedule.

C.7.1 Choose a Uucp Site Name

In a **uucp** system, every computer belongs to a given "site." A site is any computer or any Micnet network that can communicate with the **uucp** system through a modem. To distinguish one site from another, every site must have a unique "site name." A site name is any combination of letters and digits that begins with a letter and is no more than seven characters long. The site name may then be used in **uucp** and **uux** commands to direct transmissions to the appropriate computer or Micnet network.

The site name should suggest some characteristic of the site, such as its location or affiliation. For example, a site in Chicago can be named "chicago," or a site in the shipping department can be named "shipping." The site name must be unique. That is, no other computer that calls your computer or is called by your computer can have the same site name.

Once you have chosen a site name, you will need to add it to the */etc/systemid* file as described in the next section.

C.7.2 Set Up the Systemid File

Each site must have a */etc/systemid* file. The file defines the site name of the given site and associates the site with a Micnet network if any. The file has the form:

sitename
[*machinename*]

where *sitename* is the name of the given site, and *machinename* is the Micnet machine name for that computer. The machine name is optional only if the computer is not connected to a Micnet network. For example, the entries:

chicago
brewster

define a site named "chicago," whose Micnet machine name is "brewster."

Since **uucp** systems are often created after a Micnet network has been established, the *systemid* file usually exists already on a given site. In this case, you must add the site name to the beginning of each *systemid* file on each computer in the Micnet network. Note that you may give more than one machine name if desired, but each name must be on a separate line. For a full description of the *systemid* file, see **systemid**(M) in the XENIX *Reference Manual.*

You can set up the *systemid* file by choosing option 1 of the **uuinstall** program. (Refer to Section B.2.) The current site name and machine name will be displayed and you will be prompted to select a change to either of these names. If you do wish to change either or both of them, you will be prompted to enter the new names that you have chosen.

C.7.3 Create a Dial-in Site

You can create a dial-in site by installing the **uucp** login information required by other computers that wish to log in and transfer files and commands. This information consists of the following:

- One or more */etc/passwd* file entries.
- User access information in the */usr/lib/uucp/USERFILE* file.

You can create this information by using the **mkuser** command to set up login information in the */etc/passwd* file and by using the *uuinstall* command to set up the user access information. The following sections explain the required format of the information.

Once the information is installed, you can enable the system for logins by creating a dial-in line. (Refer to Section B.5.)

Create Uucp Login Entries

A dial-in site must provide a login entry for the sites that call it. These entries must be placed in the */etc/passwd* file. A **uucp** login entry has the same form as an ordinary user login entry (see the XENIX *Operations Guide*), but gives a special login directory and login program instead of the normal user directory and shell. To create a **uucp** login entry, follow these steps:

1. Choose a new login name and a user ID for the **uucp** login. The name may be any combination of letters and digits that is no more than eight characters long. Make sure the name is unique; a **uucp** login entry must not have the same name as any other login entry.

2. Use the **mkuser** command to add the password file entry for this new login name.

3. When **mkuser** prompts you for the shell to be used by this user, you should choose */usr/lib/uucp/uucico* for the shell.

Note that you can create new login entries for each site that calls your site, or use one entry for all sites.

Set Up the USERFILE

The */usr/lib/uucp/USERFILE* file defines which directories a given site (or a given user) may access using the **uucp** and **uux** commands. You should create one *USERFILE* entry for each site or user with a login entry in the */etc/passwd* file. Each entry has the form:

login,sitename pathname...

where *login* is the login name for a given site, *sitename* is the site name of a given site, and *pathname* is the full pathname of the directory the given site may access. More than one *pathname* may be given if desired. The *login* and *sitename* are optional.

The following rules explain how access is granted for each entry:

1. A calling site is granted access to those directories defined in an entry containing its site name.

2. A calling site whose name does not appear in an entry is granted access to the directories defined for the first entry with no site name.

3. A user is granted access to those directories defined in an entry containing his or her login name.

4. A user whose login name does not appear in an entry is granted access to directories defined in the first entry with no login name.

You may have more than one entry with the same login name if you wish, but you must make sure that at least one of these entries also has the site name of any calling site which can log in with that name, or that one of these entries has no site name.

For example, consider the following entries:

```
uuccg,chicago /usr /usr2/market
uucp,     /usr/vendor
schmidt, /usr2/market  /usr/vendor
,          /usr/spool/uucp/public
```

The site named *chicago* has access to files in the directories named */usr* and */usr2/market*. Any other site will be granted access to */usr/vendor* only. A local user named *schmidt* is granted access to the directories */usr2/market* and */usr/vendor*. All other users have access to */usr/spool/uucp/public* only.

This information can be set up by choosing option 5 of the **uuinstall** command. (Refer to Section B.2.) You will be prompted as to whether you need to see the current entries in the userfile. If you type "y", the screen display will be similar to the following:

```
#     Login        Sitename      Paths

0.    ANYLOGIN     ANYSITE       /tmp
1.    ANYLOGIN     microso       /tmp /usr/spool/uucp
```

The *ANYLOGIN* and *ANYSITE* entries are special entries displayed whenever a blank login name or site name field is encountered in the userfile.

You are then asked whether you wish to add or delete an entry in the file. Entries are always deleted by specifying the entry number (#) in the first column on the screen display. If you request that an entry be added, you will be prompted for the login name, site name, and pathname for the new entry. If you simply press ENTER in response to this prompt, the display will return to the main **uuinstall** menu.

In response to the requests for a login name and a site name, you may enter the special name "A" (meaning "ANY") which corresponds to a blank field in the userfile for the login or site names. The prompts for pathnames will continue until you enter a blank line.

C.7.4 Create a Dial-Out Site

You can create a dial-out site by installing the dialing information needed by your system to call and log in to other computers. This information consists of the following:

- Dialing abbreviations for remote computers in the */usr/lib/uucp/L-dialcodes* file.

- Information about logins on remote computers in the */usr/lib/uucp/L.sys* file.

- A transmission schedule in the form of a shell script to be called periodically by the **cron** program.

The following sections explain the required format of the information.

Once the information is installed, you can enable the system for calling other computers by creating a dial-out line. (Refer to Section B.6.)

Set Up the L-dialcodes File

The */usr/lib/uucp/L-dialcodes* file defines abbreviations for often-used telephone prefixes and area codes. You may use these abbreviations in the *L.sys* file when forming the telephone numbers of remote sites.

The *L-dialcodes* file may contain one or more entries of the form:

abbreviation dial-sequence

where *abbreviation* is any combination of letters and digits that begins with a letter, and *dial-sequence* is any combination of digits that represents a telephone prefix, area code, or any other part of a telephone number. For example, the entry:

ms 555

defines the abbreviation "ms" to be the telephone prefix "555".

This information can be set up by choosing option 3 of the **uuinstall** command. (Refer to Section B.2.) You will be prompted as to whether you need to see the current entries in the dialcodes file. If you type "y", the screen display will be similar to the following:

```
#     Abbreviation  Code
```

```
0.      Sea                    206
1.      London          011441
```

You are then prompted to add or delete an entry in the file. Remember that you delete entries by specifying the entry number (#) in the first column on the screen display. If you request that an entry be added, you will be prompted for the abbreviation and the dialing code for each entry. If you press ENTER in response to this prompt, the display will return to the **uuinstall** main menu.

Set Up the L.sys File

The */usr/lib/uucp/L.sys* file defines the names, telephone numbers, and login information of all sites in the system. The file contains one or more entries of the form:

sitename time device speed phone login

where *sitename* is the name of the site to be called; *time* is a combination of letters and digits that gives the weekdays and times when the given site can be called; *device* is the name of the device through which the given site is to be called; *speed* is the line speed for the call, *phone* is the phone number of the given site, and *login* is the login information required to log in to the given site. With the Hayes modem, the *speed* must be 1200.

The *time* defines when the given site can make calls to other sites. It has the form:

days times

where *days* is a list of one or more days of the week, and *times* is a range of times of day. The days of the week may be "Su", "Mo", "Tu", "We", "Th", "Fr", "Sa", "Wk" (for any weekday), "Any" (for any day), and "Never" (for call by special request only). The time of day must be given as a four digit number. The first pair of digits gives the hour (in terms of a 24 hour clock), the second pair gives the minutes. A range of times is a pair of times of the day separated by a hyphen (–). For example, the entry:

MoTuTh0800–1230

allows the given site to be called any Monday, Tuesday, or Thursday from 8 in the morning to 12:30 in the afternoon.

The *device* must be the keyword "ACU" if you are using a modem. If you are using a direct line to the other site, then you must give the filename of the serial line (or other device) you intend to use (e.g., tty01).

The *phone* must be the telephone number of the given site. It must have the correct number of digits (including area code if necessary) or be a combination of *L-dialcodes* abbreviations and digits. *L-dialcodes* abbreviations must go before any digits. Hyphens must not be used. For example, "5551234" is a valid local number and "2065551234" is a valid long distance number. If the abbreviation "ms" is defined to be "555", then "ms1234" may be used in place of "5551234".

With the Hayes modem, you may use a comma (,) in a number to cause a delay when dialing. This is useful if you must dial for an outside line before placing the call. For example, the number "9,5551234" causes a delay immediately after the "9" has been dialed. After the delay, the rest of the number is dialed. If you are not using a modem, then *phone* must be the filename of the device you intend to use instead of a phone number.

The *login* must be a sequence of names, numbers, and other information that represents the steps required to log in to the given site. This sequence has the form:

expect send [*expect send*]...

where *expect* is the prompt or message that you expect the given site to return to the calling site, and *send* is the name, number, or other information that you wish to send in response to the expected prompt or message. For example, the following is the login sequence for a typical XENIX site:

login: uuccg ssword: market

Note that "ssword:" is given instead of the complete prompt "Password:". Only the last eight characters in each expected prompt or message are examined, so you do not need to give the preceding characters if you wish to save space.

If you anticipate problems during the login sequence, you may include a conditional response immediately after each expected prompt or message. This conditional response has the form:

expect [*–send–expect1*]...

where *expect* is the prompt or message you expect the given site to return; *send* is the name or number you wish to send if the prompt or message returned is not correct, and *expect1* is the prompt or message you expect after sending the conditional response. For example, the following command line shows you how to invoke the "login" prompt if it is not immediately present:

--login-EOT-login-uuccg ssword: market

There are two special keywords that you may use in the login sequence. The EOT keyword causes an end of transmission character to be sent, and the

BREAK keyword causes a break character to be sent. (The break character is simulated using line speed changes and null characters and may not work on all devices and/or systems.)

The complete *L.sys* entry must be placed on one line as shown by the following example:

chicago Any ACU 1200 5551234 login uucp ssword: market

You can set up this information by choosing option 4 of the **uuinstall** command. (Refer to Section B.2.) You will be prompted as to whether you need to see the current entries in the *L.sys* file. If you type "y", the screen display will be similar to the following:

```
Entry #:                0
System name:            chicago
Time to call:  Any
Line:                   ACU
Speed:         1200
Phone #:                5551234
Login sequence:         login uucp ssword: market

Press Enter to see next entry
```

A new entry will be displayed each time you press the ENTER key. You are then asked whether you wish to add or delete an entry in the file. Once again, you can delete entries by specifying the entry number (#), which is in the first field for each displayed entry. If you request that an entry be added, you will be prompted for each field in turn. If you press ENTER in response to this prompt, the display will return to the **uuinstall** main menu.

The response to the prompt concerning the line to use for the call, can be either "A", which denotes an ACU, or the device number for the *tty* device to be used for the connection.

Create a Transmission Schedule

In the **uucp** system, the **uucico** program carries out all transmissions between your site and other sites, sending and receiving files and commands as long as there is work for it to do. On a dial-in site, **uucico** is always started whenever a calling site logs in, but on a dial-out site, **uucico** is only started when an explicit invocation of the program is given. This means you must invoke the program periodically on a dial-out site to ensure that all transmissions requested by the **uucp** and **uux** programs are completed. You can do this in one of two ways: invoke the program manually whenever you need it, or create a shell script and let the **cron** program invoke **uucico** automatically according to a schedule of transmissions.

The most convenient method is to let **cron** invoke **uucico** for you. To do this, you choose a schedule of times for **uucico** to be invoked, then create a */etc/crontab* file entry for this schedule. A */etc/crontab* entry has the form:

minutes hour day month day-of-week command-line

where *minutes*, *hour*, *day*, *month*, and *day-of-week* give the exact day of the year and time of day to execute the given *command-line*. Each item, except the *command-line*, must be an integer number within an acceptable range, (e.g., 0 to 59 for *minutes*). A sequence of values for one item may be given by separating the values with commas. Also, you may use an asterisk (*) to represent all acceptable values. The *command-line* must be the name of the shell script you have created to invoke **uucico**.

You can add an entry to the */etc/crontab* file by using a XENIX text editor. For more information about the file, see **cron**(C) and **crontab**(C) in the XENIX *Reference Manual*. For example, the entry:

```
15,45 * * * * /usr/lib/uucp/transmit
```

invokes the shell script "transmit" every 30 minutes to sites for which requests are pending. The entry:

```
0 0 * * * /usr/lib/uucp/transmit
```

invokes "transmit" every day at midnight, and the entry:

```
15 2,4,6 * * * /usr/lib/uucp/transmit
```

invokes the script every day at "2:15", "4:15", and "6:15" in the morning.

A shell script is simply a text file that contains one or more XENIX commands. If your **uucp** system is acting as both a dial-in and dial-out site at different times, then the script should have the form:

```
#
/bin/disable /dev/ttynn
if ($status == 0) then
        sleep 20
        /usr/lib/uucp/uucico -r1 -s sitename
        /bin/enable /dev/ttynn
endif
```

where *nn* is the number of your modem's serial line, *sitename* is the name of the site you wish to call. Use the **-s** option if you wish to force a call to the given site even if no requests for transmissions exist on the calling site. Note that you can use the **-S** option in place of the **-s** option if you wish to ignore the range of calling times given in the *L.sys* file. Give one **uucico** command for each site you wish to call. If you want to call only those sites for which requests exist, give a single **uucico** command, but do not give the **-s** or **-S** option with the command. If your computer is strictly a dial-out site, then the **enable** and **disable** commands are not required.

For example, the script:

```
#
/bin/disable /dev/tty00
if ($status == 0) then
        sleep 20
        /usr/lib/uucp/uucico -r1 -s chicago
        /bin/enable /dev/tty00
endif
```

places a call to the "chicago" site after disabling the serial line. The line must be disabled in order to dial out on that line. Similarly, it must be enabled to allow subsequent calls from other computers.

You can create a shell script by using a XENIX text editor. For convenience, the script should be placed in the */usr/lib/uucp* directory and must be given execute permissions for everyone. Note that you can also add **uucp** maintenance programs to the script. (Refer to Section B.8.7.)

C.7.5 Link Micnet Sites

To use a **uucp** system with your Micnet network, follow these steps:

1. Add the entry:

    uucp:

    to the *maliases* file of the computer on which the **uucp** system is installed.

2. For all other computers in your site, add the entry:

    uucp:*machinename*:

    to the *maliases* file. The *machinename* must be the name of the computer on which the **uucp** system is installed. This longer form of entry may also be used on the computer on which the **uucp** system is installed.

You can test the **uucp** system by mailing a short letter to yourself via another site. For example, if you are on the site "chicago," and there is another Micnet site named "seattle" in the system, then the command:

mail seattle!chicago!johnd

will send mail to the "seattle" site, then back to your "chicago" site, and finally to the user "johnd" in your Micnet network. Note that a **uucp** system usually performs its communication tasks according to a fixed schedule, and may not return mail immediately.

C.8 Maintaining the System

This section explains how to maintain the **uucp** system. In particular, it explains how to display and merge the content of **uucp** log files, how to remove old requests and files from the spool directories, and how to solve some common problems. It also includes some sample shell files, which are toward the end of the section.

You can automate some maintenance tasks by creating shell command files and initiating these files with *crontab* entries. Other tasks require manual modification.

C.8.1 Display and Merge Log Files

You can display a record of the transmissions requested and completed to a given site or user by using the **uulog** command. The command displays the contents of the individual log files created for a given site or user and merges these entries with the system log file, *LOGFILE*. The log files contain information about queued requests, calls to remote sites, execution of **uux** commands, and file copy results. The command has the form:

uulog -s*sitename* -u*user*

where **-s***sitename* names the site whose log files are to be displayed, and **-u***user* names the user whose log files are to be displayed. If you do not give a *sitename*, *user* log files for all sites and users are displayed. The command places the new log files at the beginning of the existing *LOGFILE*.

The log files are originally created in the */usr/spool/uucp* directory as individual files, but should be copied to the *LOGFILE* on a regular basis since they are not copied automatically. For example, the command:

uulog

merges all log files and displays their contents. The command:

uulog -schicago

merges only log files created for the site "chicago."

Note that the system *LOGFILE* should be removed periodically since it is copied each time new log files are put into the file.

C.8.2 Clean the Uucp Spool Directory

You can remove unwanted **uucp** system files from the **uucp** spool directory by using the **uuclean** command. The command removes temporary data, *LOG*, system status, and lock files from the spool directory if they are more than a given number of hours old. The command has the form:

uuclean –d*dir* –m –n*hours* –p*pre* –x*n*

where **–d***dir* names the directory to be scanned; **–m** causes mail to be sent to the owner of each file removed; **–n***rs* gives the age in hours of files to be removed, **–p***pre* causes files with the given prefix to be examined and removed, and **–x***n* directs the command to give the *n*th level of debugging output. You may specify up to 10 file prefixes with the **–p** option. If **–m** is given, most mail will be sent to the owner of the **uucp** programs, since most files put into the spool directory will be owned by the owner of the **uucp** programs. This is a result of the **setuid** bit being set on these programs. The default number of *hours* is 72 (3 days).

The **uuclean** program should be run once a day. You can invoke it automatically by using a system daemon such as **cron**. The command:

uuclean –pTM

removes all temporary data files that are at least three days old. The command:

uuclean –pLCK –h1 –m

removes all lock files that are at least an hour old and mails a list of each file removed to the owner.

The **uuclean** command may also be run as needed to remove unwanted files after a system crash or an aborted **uucp** program.

C.8.3 Reclaim Log Files After a Crash

You can reclaim individual log files after a system crash by changing their access mode with the **chmod** command, and then using **uulog** command. After a transmission failure or system crash, the individual log file for the transmission may be left with access mode 0222, making it impossible for the **uulog** command to read the file. To reclaim the log file, use **chmod** to change the access mode to 0666. You can then let **uulog** merge them with the *LOGFILE*.

C.8.4 Reclaim Data Files After a Crash

You can check the status of files transmitted from a remote site and possibly reclaim some or all of the data lost during an aborted transmission by examining system data files. The data files contain the contents of files copied from remote sites. These files are kept temporarily in the */usr/spool/uucp* directory and their names have the form:

TM.*pid.ddd*

where *pid* is a process-id, and *ddd* is a sequential three digit number starting at zero for each invocation of **uucico** and incremented for each file received.

The temporary data files are normally moved to the requested destination immediately after the transmission finishes. However, if a transmission fails or the system crashes, the file remains in the spool directory. You can examine the contents of this file with the **cat** command. If desired, you can reclaim the file by moving it to a new location with the **mv** command. Leftover data files that cannot be reclaimed should be removed using the **uuclean** command.

C.8.5 Check the Transmission Status

You can check the status of transmissions between sites in the **uucp** system by examining the system status files. System status files contain information about login, dialup, or sequence check failure, as well as the talking status when two machines are conversing. The files are kept in the */usr/spool/uucp* directory and their names have the form:

STST.*sitename*

where *sitename* is the name of the remote site.

Normally, system status files are removed after each successful transmission, but when a failure occurs, the **uucp** system copies information about the failure to the file and leaves it in the directory. This prevents the **uucp** system from making further calls to the given site for about an hour, or for sequence check failures, until the file is removed.

To examine the status, use the **cat** command to display the contents of the file. If problems with transmissions are detected, there may be a problem with the modem or with the serial line connected to the modem.

If a system status file has been left due to a program or system crash, the file may prevent all subsequent transmissions to the given site. In this case, you must remove the file before attempting further calls.

C.8.6 Check For Locked Sites or Devices

You can make sure the **uucp** system is not intentionally preventing transmissions to a given site or through a given device by examining the system lock files. The **uucp** system creates a lock file for each site being called and for each device being used to call a site. Lock files prevent the **uucp** system from attempting to duplicate conversations with a given site or from placing multiple calls on the same device. The lock files are kept in the */usr/spool/uucp* directory and their names have the form:

LCK..*str*

where *str* is either a site name or the name of the calling device.

Since lock files prevent all calls to a given site or through a given device; it is wise to make sure no unnecessary lock files are left in the directory. If a transmission has been aborted or the system has crashed, the lock files will prevent subsequent transmissions for about about 24 hours. If you wish to place a call before this time, you must remove the file using the **uuclean** command.

C.8.7 Create Maintenance Shell Files

The **uulog** and **uuclean** command can be invoked automatically by placing the commands in a shell file and creating a *crontab* entry for the shell file. The system daemon, **cron**, will then invoke the commands at the given times and most of the simple maintenance will be performed. For example, you can create a shell file that removes *TM*, *ST*, and *LCK* files daily. You can also create a shell file to maintain *C.* or *D.* files for work which can not be accomplished for such reasons as bad phone number and login changes. In this case, the shell file should contain the following commands:

```
/usr/lib/uucp/uuclean -pTM -pC. -pD.
/usr/lib/uucp/uuclean -pST -pLCK -n12
```

Note that the **-n12** option causes any *ST* and *LCK* files that are older than 12 hours to be deleted. An appropriate *crontab* entry must be created in order to invoke the shell file automatically.

C.9 Details of Operation

This section describes the details of **uucp** system program operation. It explains the processes used to create system communication and defines the files used to support the system.

C.9.1 Uucp Programs

The **uucp** system consists of four primary and two secondary programs. The primary programs are:

uucp This program creates work and gathers data files in the spool directory for the transmission of files.

uux This program creates work and execute files, and gathers data files for the remote execution of XENIX commands.

uucico This program executes the work files for data transmission.

uuxqt This program executes XENIX commands found in execution files.

The secondary programs are:

uulog This program updates the log file with new entries and reports on the status of **uucp** requests.

uuclean This program removes old files from the spool directory.

dial This program directs the modem to dial a remote site.

C.9.2 Uucp Directories and Files

During execution of the **uucp** programs, the **uucp** system uses files from the following three directories:

/usr/lib/uucp This is the directory used for **uucp** system files and all executable programs other than **uucp** and **uux**.

/usr/spool/uucp This is the spool directory used during **uucp** execution.

/usr/spool/uucp/.XQTDIR
This directory is used during execution of execute files.

Files are created in a spool directory for processing by the **uucp** daemons. There are three types of files used for the execution of work:

Data files Contain data for transfer to remote sites.

Work files Contain directions for file transfers between sites.

Execution files	Contain directions for XENIX command executions which involve the resources of one or more sites.

C.9.3 Uucp – Site to Site File Copy

The **uucp** program is the user's primary interface with the system. The **uucp** program was designed to look like the **cp** command. The syntax is:

uucp [*option*]... *source* ... *destination*

where *source* and *destination* may contain the *sitename!* prefix which indicates the site on which the file or files reside or where they will be copied.

The options interpreted by **uucp** are:

–d	Make directories when necessary for copying the file.
–c	Don't copy source files to the spool directory, but use the specified source when the actual transfer takes place.
–m	Send mail on completion of the work.

The following options are used primarily for debugging:

–s*dir*	Use the *dir* directory for the spool directory.
–x*num*	Use *num* as the level of debugging output.

The destination may be a directory name, in which case the filename is taken from the last part of the source's name. The source name may contain special shell characters such as "?*[]". If a source argument has a *sitename!* prefix for a remote site, the filename expansion will be done on the remote site. For example, the command:

uucp *.c chicago!/usr/dan

sets up the transfer of all files whose names end with *.c* to the */usr/dan* directory on the "chicago" machine.

The source and/or destination names may also contain a ~*user* prefix. This prefix translates to the login directory on the specified site. For names with partial pathnames, the current directory is prepended to the filename. Filenames with "../" are not permitted.

The command:

uucp chicago!~dan/*.h ~dan

sets up the transfer of files whose names end with *.h* in *dan*'s login directory on site, *usg*, to *dan*'s local login directory.

For each source file, the program will check the source and destination filenames and the site-designator of each to classify the work into one of five types:

1. Copy source to destination on local site.

2. Receive files from other sites.

3. Send files to remote sites.

4. Send files from remote sites to another remote site.

5. Receive files from remote sites when the source contains special shell characters as mentioned above.

After the work has been set up in the spool directory, the **uucico** program must be started to try to contact the other machine to execute the work.

Copy Files to a Local Destination

A **cp** command is used to do type 1 work. The **-d** and the **-m** options are not honored in this case.

Receive Files from Other Sites

For type 2 work, a one line work file is created for each file requested, and is put in the spool directory with the following fields, each separated by a blank:

[1] R

[2] The full pathname of the source or a ~user/pathname. The ~*user* part will be expanded on the remote site.

[3] The full pathname of the destination file. If the ~*user* notation is used, it will be immediately expanded to be the login directory for the user.

[4] The user's login name.

[5] A "–" followed by an option list. (Only the **-m** and **-d** options will appear in this list.)

Send Files to Remote Sites

For type 3 work, a work file is created for each source file and the source file is copied into a data file in the spool directory. (A **-c** option on the **uucp** program will prevent the data file from being made. In this case, the file will be transmitted from the indicated source.) Pathnames are checked using the *USERFILE* to verify access to the requested directory. The fields of each entry are as follows:

[1]	S
[2]	The full pathname of the source file.
[3]	The full pathname of the destination or ~ user/filename.
[4]	The user's login name.
[5]	A "–" followed by an option list.
[6]	The name of the data file in the spool directory.
[7]	The file mode bits of the source file in octal print format (e.g., 0666).

Copy Files Between Sites

For type 4 and 5 work, **uucp** generates a **uucp** command line and sends it to the remote machine; the remote **uucico** executes the command line.

C.9.4 Uux – Site To Site Execution

The **uux** command is used to set up the execution of a XENIX command where the execution machine and/or some of the files are remote. The syntax of the **uux** command is:

uux [–] [*option*]... *command-string*

where *command-string* is made up of one or more arguments. All special shell characters such as "<>|^" must be quoted either by quoting the entire command string, or by quoting the character as a separate argument. Within the command string, the commands and filenames may contain a *sitename!* prefix. Any arguments which do not contain a ! will not be treated as files. (They will not be copied to the execution machine.) The – option is used to indicate that the standard input for the given command should be inherited from the standard input of the **uux** command. The only option is essentially for debugging: **-x***num* directs the command to use *num* as the level of debugging output.

The command:

pr abc | uux – chicago!rmail joe

will set up the output of "pr abc" as standard input to a **mail** command to be executed on site *usg*.

Uux generates an execute file which contains the names of the files required for execution (including standard input), the user's login name, the destination of the standard output, and the command to be executed. This file is either put in the spool directory for local execution or sent to the remote site using a generated send command (type 3 above).

For required files which are not on the execution machine, **uux** will generate receive command files (type 2 above). These command files will be put on the execution machine and executed by the **uucico** program. (This will work only if the local site has permission to put files in the remote spool directory as controlled by the remote *USERFILE*.)

The execute file will be processed by the **uuxqt** program on the execution machine. It is made up of several lines, each of which contains an identification character and one or more arguments. The order of the lines in the file is not relevant and some of the lines may not be present. Each line is described in the following paragraphs.

User Line

U *user site*

where the *user* and *site* are the requester's login name and site.

Required File Line

F*filename real-name*

where the *filename* is the generated name of a file for the execute machine and *real-name* is the last part of the actual filename (contains no path information). Zero or more of these lines may be present in the execute file. The **uuxqt** program will check for the existence of all required files before the command is executed.

Standard Input Line

I *filename*

The standard input is either specified by a < in the command string or inherited from the standard input of the **uux** command if the – option is used. If a standard input is not specified, */dev/null* is used.

Standard Output Line

O*filename sitename*

The standard output is specified by a > within the command string. If a standard output is not specified, */dev/null* is used. (Note that the use of >> is not in effect.)

Command Line

C *command* [*arguments*]...

The *arguments* are those specified in the command string. The standard input and standard output will not appear on this line. All required files will be moved to the execution directory (a subdirectory of the spool directory) and the XENIX command is executed using the shell. In addition, a shell PATH statement is prepended to the command line as specified in the **uuxqt** program.

After execution, the standard output is copied or set up to be sent to the proper place.

C.9.5 Uucico – Copy In, Copy Out

The **uucico** program will perform the following major functions:

- Scans the spool directory for work.
- Places a call to a remote site.

- Negotiates a line protocol to be used.
- Executes all requests from both sites.
- Logs work requests and work completions.

Uucico may be started by a system daemon, by the user (this is usually for testing), or by a remote site. (The **uucico** program should be specified as the shell field in the */etc/passwd* file for **uucp** logins.)

When started with the **–r1** option, the program is considered to be in MASTER mode. In this mode, a connection will be made to a remote site. If started by a remote site, the program is considered to be in SLAVE mode.

The MASTER mode will operate in one of two ways. If no site name is specified (the **–s** option not specified), the program will scan the spool directory for sites to call. If a site name is specified, that site will be called, and work will only be done for that site.

The **uucico** program must generally be started directly by the user or by another program, such as a shell script invoked by **cron**. There are several options used for execution:

–r1 Starts the program in MASTER mode. This is used when **uucico** is started by a program or **cron** shell.

–s*sitename* Works only for site *sitename*. If **–s** is specified, a call to the specified site will be made even if there is no work for the site in the spool directory, but will only call when times in the *L.sys* file permit it. This is useful for polling sites which do not have the hardware to initiate a connection.

–S*sitename* Works only for site *sitename*. If **–S** is specified, a call to the specified site will be made even if there is no work for the site in the spool directory. Unlike **–s**, this option ignores the call times for the *sitename* given in the *L.sys* file.

The following options are used primarily for debugging:

–d*dir* Uses directory *dir* for the spool directory.

–x*num* Uses *num* as the level of debugging output.

The next part of this section describes the major steps within the **uucico** program.

Scan For Work

The names of the work-related files in the spool directory have the following format:

type. sitename grade number

where *type* may be "C" for copy command file, "D" for data file, "X" for execute file; *sitename* is the remote site; *grade* is a character, and *number* is a four digit, padded sequence number. For example, the file:

C.res45n0031

is a work file for a file transfer between the local machine and the "res45" machine.

The scan for work is done by looking through the spool directory for work files (files with prefix "C."). A list is made of all sites to be called. **Uucico** calls the site specified by the **–s** or **–S** option and processes the corresponding work files.

Call a Remote Site

The call is made using information from several files which reside in the **uucp** program directory. At the start of the call process, a lock is set to forbid multiple conversations between the same two sites. The lock filename has the form:

LCK..*str*

where *str* is the device name. The file is in the */usr/spool/uucp* directory.

The site name is in the *L.sys* file. The information contained for each site is:

[1] Site name.

[2] Times to call the site (days-of-week and times-of-day).

[3] Device or device type to be used for call.

[4] Line speed.

[5] Phone number if field [3] is "ACU", or the device name (same as field [3]), if not.

[6] Login information (multiple fields).

The time field is checked against the present time to see if the call should be made.

The *phone number* may contain abbreviations (e.g., mh, py, boston) which get translated into dial sequences using the *L-dialcodes* file.

The *L-devices* file is scanned using device type and line speed fields from the *L.sys* file to find an available device for the call. The program will try all devices which satisfy these fields until the call is made or until no more devices can be tried. If a device is successfully opened, a lock file is created so that another copy of **uucico** will not try to use it. If the call is complete, the login information in the last field of *L.sys* is used to log in.

The conversation between the two **uucico** programs begins with a handshake started by the SLAVE site. The SLAVE program sends a message to let the MASTER program know it is ready to receive the site identification and conversation sequence number. The response from the MASTER is verified by the SLAVE, and if acceptable, protocol selection begins. The SLAVE program can also reply with a call-back required message in which case, the current conversation is terminated.

Select Line Protocol

The remote site sends a message:

P*proto-list*

where *proto-list* is a string of characters, each representing a line protocol.

The calling program checks the protocol list for a letter corresponding to an available line protocol and returns a use protocol message. The message has the form:

U*code*

where *code* is either a one character protocol letter or "N" which means there is no common protocol.

Process Work

The initial role of MASTER or SLAVE to the processing work is the mode in which each program starts. (The MASTER has been specified by the **-r1** option.) The MASTER program does a work search similar to the one used in the section, "Scan For Work," above.

There are five messages used during the process work, each specified by the first character of the message. They are:

S Send a file.

R Receive a file.

C Copy complete.

X Execute a **uucp** command.

H Hangup.

The MASTER program will send ''R'', ''S'', or ''X'' messages until all work from the spool directory is complete, at which point an ''H'' message is sent. The SLAVE program will reply with the first letter of the request and either the letter ''Y'' or ''N'' for yes or no. For example, the message ''SY'' indicates that it is okay to send a file.

The send and receive replies are based on permission to access the requested file/directory using the *USERFILE* and read/write permissions of the file/directory. After each file is copied into the spool directory of the receiving site, a copy-complete message is sent by the receiver of the file. The message ''CY'' will be sent if the file has successfully been moved from the temporary spool file to the actual destination. Otherwise, a ''CN'' message is sent. (In the case of ''CN'', the transferred file will be in the spool directory with a name beginning with ''TM''.) The requests and results are logged on both sites.

The hangup response is determined by the SLAVE program using a work scan of the spool directory. If work for the remote site exists in the SLAVE's spool directory, an ''HN'' message is sent and the programs switch roles. If no work exists, an ''HY'' response is sent.

Terminate a Conversation

When an ''HY'' message is received by the MASTER program it is echoed back to the SLAVE program and the protocols are turned off. Each program sends a final ''OO'' message to the other. The original SLAVE program will clean up and terminate. The MASTER program will proceed to call other sites and process work as long as possible or terminate if a **–s** option was specified.

C.9.6 Uuxqt – Uucp Command Execution

The **uuxqt** program is used to process execute files generated by **uux**. The **uuxqt** program is started by the **uucico** program. The program scans the spool directory for execute files (prefix *X.*). Each one is checked to see if all the required files are available, and, if so, the command line or send line is executed by the following shell command:

sh –c

which you use with the command line after you have opened appropriate standard input and standard output. If a standard output is specified, the program will create a send command or copy the output file as appropriate.

C.9.7 Security

In unrestricted **uucp** systems, once users log in to another site through the **uucp** system, they can execute any commands and copy any files normally accessible to the **uucp** login. It is up to the individual sites to be aware of this and apply the protections that they feel are necessary to prevent unauthorized use of files and commands.

The **uucp** system does provide a certain level of security. For example, a calling site does not get a standard shell when it logs in. Instead, the **uucico** program is started and all work is done through this program. The **uucico** program checks the pathnames of files to be sent or received to prevent access to restricted directories. The *USERFILE* supplies the information for these checks. To prevent execution of possibly damaging commands, the **uuxqt** program can only execute the **rmail** program on a remote site. This special program is one of many underlying mail programs that help deliver mail. Finally, the *L.sys* file is owned by **uucp** and has mode 0400 to protect the phone numbers and login information for remote sites.

C.10 Creating a New Dial Program

The **dial** program is used by dial-out sites to place calls to other computers. You can create a new **dial** program for any dial-out site not using the Hayes Smartmodem by modifying the source program and compiling it with the **cc** command. See the **cc**(CP) command in the XENIX *Reference Manual* for more information. The **dial** source program is included on the following pages.

```
/*
 *
 *      Copyright (C) Microsoft Corporation, 1983
 *
 * Simple dialer program for the Hayes "Smart" Modem 1200
 *
 *      See Hayes Reference Manual for command definitions
 *
 * Usage: dial ttyname telnumber speed
 *
 *      returns 0 if a connection was made
 *              -1 otherwise
 */

#include <stdio.h>
#include <signal.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/ioctl.h>
#include <termio.h>

#define SAME    0
char *setup = "M1 F1 DT";          /* Speaker on, Full Duplex, Touch tone*/
struct termio term;
int baudrate;                              /* baud rate of modem */
char buffer[80];
int alrmint();

main(argc,argv)
int argc;
char *argv[];
{
        FILE *fdr,*fdw;
        int fd;

        if( argc != 4) {
                fprintf(stderr,"Usage: dial  devicename [number] speed\n");
                exit(-1);
        }
        if( (fd=open(argv[1],O_RDWR|O_NDELAY)) < 0 ) {
                fprintf(stderr,"dial: Can't open device: %s for reading.\n",argv[1]);
                exit(-1);
        }
        switch(atoi(argv[3])) {
                case 300:
                        baudrate = B300;
                        break;
                case 1200:
```

```
                        baudrate = B1200;
                        break;
                default:
                        baudrate = B1200;
        }
        /*
         * set line for no echo and specific speed
         */
        ioctl(fd, TCGETA, &term);
        term.c_cflag &= ~CBAUD;
        term.c_cflag |= CLOCAL|HUPCL|baudrate;
        term.c_lflag &= ~ECHO;
        term.c_cc[VMIN] = 1;
        term.c_cc[VTIME] = 0;
        ioctl(fd, TCSETA, &term);
        fcntl(fd, F_SETFL, fcntl(fd, F_GETFL, 0) & ~O_NDELAY);
        if( (fdr=fopen(argv[1],"r")) == NULL ) {
                fprintf(stderr,"dial: Can't open device: %s for reading.\n",argv[1]);
                exit(-1);
        }
        if( (fdw=fopen(argv[1],"w")) == NULL ) {
                fprintf(stderr,"dial: Can't open device: %s for writing.\n",argv[1]);
                exit(-1);
        }
        setbuf(fdw,0);                          /* Want unbuffered I/O */
        /*
         * setup for timeout in 10 seconds if no response
         */
        signal(SIGALRM, alrmint);
        alarm(10);
reread:
        fprintf(fdw,"AT\r");                    /* Put Hayes into command mode */
        if( fgets(buffer,sizeof buffer, fdr) == NULL )
                exit(-1);
        if( strncmp(buffer, "OK",2) != SAME ) {         /* got back an OK? */
                sleep(1);
                goto reread;
        }
        alarm(0);                                       /* turn off alarm */
        sleep(1);
        fprintf(fdw,"AT %s %s\r",setup,argv[2]);        /* put out dialing string */
        /*
         * turn off CLOCAL now, since we want modem interrupts to work
         * setup alarm. (Longer timeout period for longer numbers)
         *
        ioctl(fd, TCGETA, &term);
        term.c_cflag &= ~CLOCAL;
        ioctl(fd, TCSETA, &term);
```

```
        */
        alarm((4*strlen(argv[2])) + 5);
again:
        if( fgets(buffer,sizeof buffer,fdr) == NULL )
                        exit(-1);
        if( strncmp(buffer, "NO CARRIER",10) == SAME ) {
                exit(-1);
        }
        if( strncmp(buffer, "CONNECT",7) != SAME ) {
                goto again;
        }
        exit(0);
}

alrmint()
{
        exit(-1);
}
```

Index - Operations Guide